# Software Design Document (SDD)
# Assignment #2 Two-Pass SIC/XE Assembler
# CS530

**Team:**
Kristen Waterford, cssc2556, Red ID: 826655811
Tasks:

- Program design and pseudocode for two-pass assembler logic
- Parser.h / Parser.cpp — line parsing, label/opcode/operand/comment field detection, comment line handling
- Pass 1 implementation — LOCCTR management, SYMTAB construction, directive handling (START, END, RESW, RESB, BYTE, WORD, BASE, NOBASE)
- Literal collection in pass 1 (pendingLits, LITTAB population at */LTORG/END)
- Unit testing of parser and pass 1 address assignment

Duncan Hugelmaier, cssc2523, Red ID: 131941547
Tasks:

- OpcodeTable.h / OpcodeTable.cpp — SIC/XE opcode table design and implementation
- Pass 2 implementation — object code generation for all formats (1, 2, 3, 4), nixbpe flag encoding, PC-relative and base-relative displacement calculation
- parseOperandMode() helper — addressing mode parsing (#, @, ,X), symbol/literal resolution
- writeListingFile() / writeSymtabFile() — output file formatting
- Makefile and build system configuration
- Integration testing and debugging with sample files
- Configuration management (Git)

**Team Organization and Effectiveness**

The team was divided by component responsibility. Kristen owned the Parser, pass 1 logic, and symbol table construction. Duncan owned the OpcodeTable, pass 2 object code generation, and output file formatting. Both members reviewed each other's code before integration.

Communication was handled through messaging and meetings during the design and debugging phases. The division of work was effective — the components had clear interfaces (SourceLine struct, symtab, littab) that allowed parallel development and straightforward integration.

An area for improvement would be establishing earlier integration checkpoints. Some interface assumptions between pass 1 and pass 2 (such as literal pool line format) required back-and-forth adjustments during integration that could have been caught sooner with a shared integration test.

**Overview & Goals:**
The purpose of this project is to design and implement a two-pass assembler for the SIC/XE architecture.

**Goals:**

- Parse .sic source program
- Build a symbol table (SYMTAB) during Pass 1
- Generate object code during Pass 2
- Listing file (.l)
- Symbol table file (.st)
- Handle addressing modes and directives correctly
- Ensure robust error handling and efficient processing

**Project Description:**

The purpose of this project is to develop a limited two-pass assembler for the XE variant of the SIC/XE family of machines. The assembler reads SIC/XE source files (.sic), processes each line through two passes, and produces a formatted listing file (.l) and a symbol table file (.st) for each input.

The program accepts one or more source filenames as command-line arguments. If no arguments are provided, it prints a usage message and exits. For each file, the assembler runs pass 1 to assign addresses and build the symbol table, then pass 2 to generate object code using PC-relative, base-relative, or direct addressing as appropriate. Errors detected during assembly are flagged inline in the listing file. The program does not implement EQU, ORG, or object program file generation.

**Plan of Action and Milestones:**

Phase 1: Design and Planning (March 9 - March 12)

Review SIC/XE architecture, instruction formats, and assembler directives through section 2.3.4
Identify required data structures: SourceLine struct, symtab, littab, pendingLits
Define component responsibilities and divide team tasks
Design pseudocode for pass 1 and pass 2 logic
Draft initial SDD

Phase 2: Parser and Pass 1 (March 13 – March 20)

Implement Parser::parseLine() — label detection, opcode/operand/comment field splitting, comment line handling
Implement pass 1 — LOCCTR initialization from START, symbol table construction, instruction length calculation for all formats
Implement directive handling: RESW, RESB, BYTE, WORD, BASE, NOBASE, END
Implement literal collection and LITTAB population at */LTORG/END
Test pass 1 output against the sample listing file addresses

Phase 3: Opcode Table and Pass 2 (March 21 – April 10)

Implement OpcodeTable.cpp with all 59 SIC/XE mnemonics mapped to opcode bytes
Implement parseOperandMode() — strip #/@/,X, resolve symbols and literals to addresses
Implement pass 2 object code generation:
Format 1: 1-byte opcode
Format 2: opcode + register nibbles
Format 3: nixbpe flags, PC-relative then base-relative displacement
Format 4: nixbpe flags with e=1, full 20-bit address
Implement RSUB, WORD, BYTE, and literal pool (*) encoding

## Phase 4: Output and Testing (April 11 – April 19)

Implement writeListingFile() — formatted columns: line number, address, object code, source text, error messages
Implement writeSymtabFile() — symbol name and hex address
Test full assembly of P2sample.sic against expected listing output
Debug displacement calculations and addressing mode flags
Verify BASE/NOBASE tracking in pass 2

## Phase 5: Finalization (April 20 – Submission Date)

Add team member names and Red IDs to all source file headers
Complete and finalize SDD
Write and verify README with build/run instructions
Final testing on Edoras with make and sample input files
Package and submit tarball

**Functional and System Requirements:**

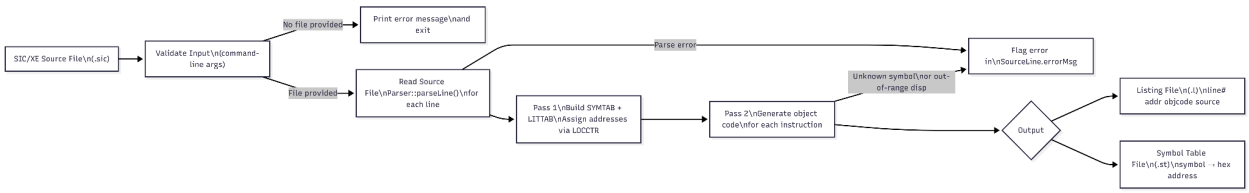
### Functional:

- Read and parse a .sic source file line by line
- Parse each source line into label, opcode, operand, comment fields
- Pass 1: Build symbol table (SYMTAB), assign addresses via LOCCTR, collect literals into LITTAB
- Pass 2: Generate object code for each instruction and directive using SYMTAB and LITTAB
- Support all SIC/XE instruction formats (1, 2, 3, 4)
- Support addressing modes: simple, immediate (#), indirect (@), indexed (,X), PC-relative, base-relative
- Support assembler directives: START, END, WORD, RESW, RESB, BYTE, BASE, NOBASE, LTORG
- Write a formatted listing file (.l) with line numbers, addresses, object code, and source
- Write a symbol table file (.st) with all symbols and their addresses
- Report errors inline in the listing file

### System Requirements:

- The program must be written in C++.
- The program must compile successfully using the GCC compiler.
- The program must run on macOS or Windows operating systems.

**System Design/Specification**




**Pseudocode for Pass 1 & 2:**

```
Pass 1:
  locctr ← START address (or 0)
  for each source line:
    if comment → skip
    record line.address ← locctr
    if label present → add to SYMTAB (error if duplicate)
    if operand starts with '=' → add to pendingLits if not already there
    if opcode is START  → set locctr
    if opcode is WORD   → locctr += 3
    if opcode is RESW   → locctr += 3 * n
    if opcode is RESB   → locctr += n
    if opcode is BYTE   → locctr += byte length of operand
    if opcode is BASE/NOBASE → no change
    if opcode is LTORG or * → flush pendingLits into LITTAB, advance locctr
    if opcode is END    → flush pendingLits, stop
    if opcode is instruction → locctr += format length (1/2/3/4)
```

```
Pass 2:
  for each source line:
    if comment or has error → skip
    if BASE  → record baseAddress
    if NOBASE → clear baseAddress
    if START/END/RESW/RESB/LTORG → skip
    if * (literal pool line) → encode literal value as hex bytes
    if WORD → encode 24-bit value
    if BYTE → encode C'' as ASCII hex or X'' as raw hex
    if instruction:
      look up mnemonic in OPTAB
      if format 1 → emit 1-byte opcode
      if format 2 → emit opcode + r1 nibble + r2 nibble
      if format 3/4:
        call parseOperandMode() → get n,i,x flags and targetAddr
        if format 4 (+prefix) → emit 4 bytes with e=1, 20-bit address
        else:
          try PC-relative: disp = targetAddr - (PC+3), range [-2048, 2047]
          else try base-relative: disp = targetAddr - base, range [0, 4095]
          emit 3 bytes with nixbpe flags and displacement
```

## Data Structures

**SourceLine struct** — holds lineNumber, label, opcode, operand, comment, address, objectCode, errorMsg for each parsed line

**symtab (map<string,int>)** — symbol name → hex address
**littab (map<string,int>)** — literal string → assigned address
**pendingLits (vector<string>)** — literals collected but not yet assigned addresses

Component Descriptions:
**Parser** — statically parses a raw source line: detects comment lines (. prefix), handles label detection (non-space in column 0), splits tokens into fields
**OpcodeTable** — provides a singleton map of all 59 SIC/XE mnemonics to their opcode bytes
**Assembler::pass1()** — scans all lines, maintains LOCCTR, populates SYMTAB and LITTAB, validates directives
**Assembler::pass2()** — encodes each line: computes nixbpe flags, calculates PC-relative or base-relative displacements, formats object code as uppercase hex strings
**parseOperandMode()** — helper that strips #/@/,X prefixes/suffixes, resolves symbols/literals to addresses, returns OperandInfo struct

**Development Environment:**

- Operating System: Windows and MAC OS
- IDE: VSCode
- Compiler: GCC Compiler
- Edoras from SDSU
- GitHub (version control)
- GDB debugger
- C++ standard libraries used: <map>, <vector>, <fstream>, <sstream>, <iomanip>

**Tools and Libraries**

**IDE**: Visual Studio Code
**Compiler**: GCC g++ (C++17), tested on Edoras (SDSU Linux servers)
**Version Control**: Git / GitHub — used for source management and collaboration
**Remote Testing**: Edoras via SSH/SCP

**C++ Standard Libraries Used:**
**<fstream>** — file input/output for .sic, .l, .st files
**<sstream>** — string stream for building hex object code strings
**<iomanip>** — hex formatting and field-width control in listing output
**<map>** — SYMTAB, LITTAB, OPTAB, REGTAB
**<vector>** — source line storage, pending literal list
**<cctype>** — character classification for uppercase conversion and digit checking

**Run/Test Environment:**

The project will be developed and tested using the VSCode IDE. We will write the program in C++ and it will be compiled with the standard g++ compiler. Version control will be handled with a GitHub repository. Testing will be performed using multiple input files to test different cases and will be done both locally from the command line and on Edoras.

**Code and Test Plan:**

Code Review:
Group members will review the source code for completion and correctness, including: correct LOCCTR advancement for all instruction formats and directives, accurate symbol table population during pass 1, correct

nixbpe flag encoding and PC/base-relative displacement calculations in pass 2, and proper error flagging in the listing file.

### Unit Testing:

Ensure every program requirement is met. Each method will be tested individually — for example, Parser::parseLine() will be verified against lines with and without labels, getByteLength() against both C and X operand types, and parseOperandMode() against immediate, indirect, indexed, and literal operands.

### Integration Testing:

Combine all source files into one package. Test the program with P2sample.sic and verify the .l listing addresses and object codes match the expected output from the textbook. Additional edge-case input files will be used to test error handling, unresolved symbols, and out-of-range displacements.